MIT 8103 Advanced Database Systems: Portfolio Summary

Veracore MFB digital bank | Emmanuel Awa | Student ID: 301914400

GitHub: awaemma/advanced-database-portfolio

Case study

Veracore MFB is a microfinance bank in Nigeria that utilizes an app, website, and USSD for their services. Customer savings accounts and wallets are used through this system. Veracore's customers, their accounts, transfers to and from other banks as well as internally to Veracore MFB, the audit trail of transactions, and security event logs like login attempts that fail, all exist within the system. The main database is PostgreSQL. Data is replicated on my personal MongoDB in a Docker instance. All customer and transaction data is fake. This model represents a small fraction of a real bank.

Portfolio 1: Database Design and Modelling

There are 10 tables in the relational model. These tables are: customer, account, channel, external_institution, session_registry, the three transfer tables (outgoing transfer, incoming transfer and internal transfer), ledger_entry and audit_event. As far as storage goes, there are no balances. There are however ledger entries and audit events. Credits minus debits equal balances. Therefore, the ledger entry of the transfer reversal will be the last one, and the rest will remain untouched. Each transfer is given a session ID, which is a number up to 30 digits in length, that needs to be unique. In this particular case, each unique session ID is inserted into the session_registry because a UNIQUE constraint can only be enforced in a single table. All accounts are identified by their unique 10-digit account number. Ledger_entry is partitioned by month using the posted_at column and the different application, operations and audit database roles control access to the tables and data. I started normalization by looking at an outward transfer flat report. I placed all repeating ledger entries in their own rows for 1NF, and moved all the partial dependencies that belonged to session ID in 2NF, as well as the transitive dependencies like channel name in 3NF, customer name through customer, etc. All the final tables have BCNF. There are 500 customers in the database along with 874 accounts and 100,000 transfers and 155,090 ledger entries. I ran 19 constraint tests and 7 access tests, all of which had the expected result. Frozen accounts are not prohibited by the schema nor is debiting an account below a zero balance, which I noted in the README..

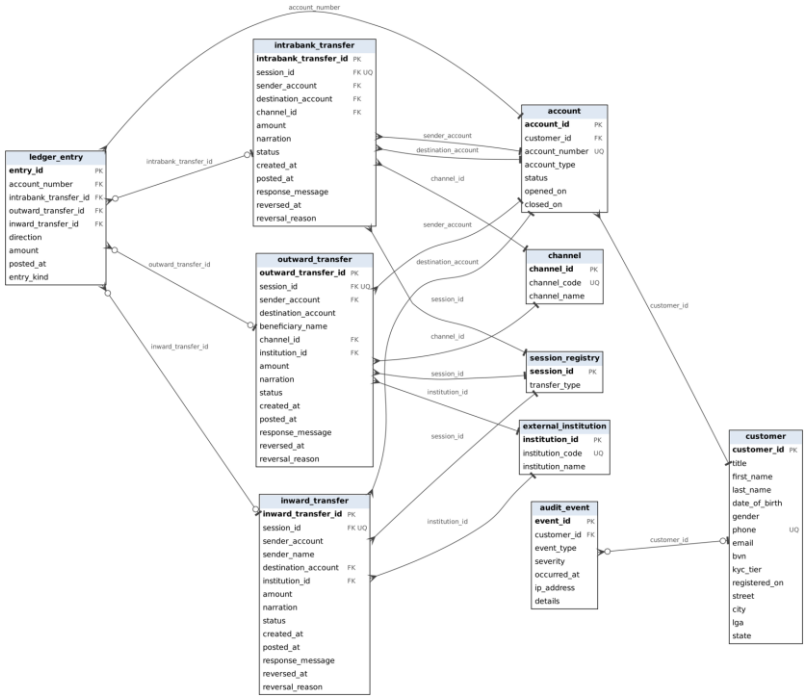


Figure 1. The ten tables and their foreign keys. The full-size diagram, the two ER diagrams and the schema script are in Portfolio 1.

Portfolio 2: Query Processing and Optimisation

I ran four queries on the seeded data with EXPLAIN (ANALYZE, BUFFERS), first without and then with five added indexes: an account statement over two partitions, failed transfers by channel and day, the top ten accounts by debit volume, and the latest 20 transfers for one account. Each query was run twice and I kept the second run, so the data was already cached.

Query	Time before (ms)	Time after (ms)	Buffers before	Buffers after
Statement for one account	160.125	0.202	2,148	88
Failed transfers by channel and day	16.078	6.992	1,918	1,711
Top 10 accounts by debit volume	20.158	21.168	504	504
Latest 20 transfers for one account	7.472	0.117	1,393	26

The indexes helped most where a query picks a few rows out of a large table. The failed-transfer query still reads each failed row from the table, so it improved less. The top-ten query reads most of its partition and kept the same plan, so the covering index was not used and the time did not improve. Each index also adds work to every insert and takes disk space.

Portfolio 3: Transactions and Concurrency

The transfer of 5,000.00 was made from two accounts, and was committed with both accounts' registry rows and the transfer row, as well as both accounts' ledger entries. A second transfer failed on its last insert, because the direction value refund is a violation of a CHECK constraint; the ROLLBACK command has also rolled back the inserts prior to the error. SHOW determined the default isolation level is READ COMMITTED.

In that session of pgAdmin, each user took out 15,000.00 from an account. The second session had read the balance before the first one had committed, and after that it queried the balance, and its output was -8,232.60. I rolled that session back as it would make my data invalid. Both sessions issued SELECT ... FOR UPDATE to lock the account row, in the second session they waited, saw the new balance, and stopped, and the account balance remained unchanged with only one account withdrawal. This protection is provided by the nature of the transaction itself, and not by the schema. A last test recorded 2,000 audit events, then recorded another 2,000 events when it was committed, and then recorded 2,001 events after committing.

Portfolio 4: NoSQL and Advanced Data Models

There are three collections in MongoDB from the PostgreSQL data: customers, with accounts embedded; transfers, with ledger entries embedded (the session ID is the _id); and audit events, which contain a free-form details field. The ledger remains in PostgreSQL because it requires multi-row transactions and the checks for Portfolio 3. There are validation rules for each collection. Seven bad inserts were all rejected and the counts did not change. The load included 500 customers, 100,003 transfers and 2001 audit events (including the rows added in the Portfolio 3 tests).

One document was examined by finding customer by account number, and the last 20 transfers on an account were examined. For the top-ten pipeline there was no usable index, a scan of all 100,003 documents and a time of 331 ms. The same thing in PostgreSQL lasted approximately 20ms. A fair comparison is not that because the data is different and MongoDB is running in Docker but that's one reason I opted to keep the ledger in PostgreSQL. The MongoDB copy is only reloaded and does not synchronize with PostgreSQL.

Portfolio 5: Distributed and Cloud Database Exercise

I used a 3 node MongoDB replica set with Docker. A document written on the primary was read back from a secondary and the lag reported was as follows: 0 seconds on an idle set. I then stopped the primary. After 15 seconds, it was primary and accepted a write and the old primary was restarted and became the secondary with both documents. A backup of the Veracore database was restored into a second database containing all 102,504 documents and indexes and the counts were the same.

Sharding and the cloud deployment are just discussed. For the transfers I would shard on sender_account and created_at [6]. Sharding by created_at will route any new write to any one of the shards, while a hash on the session will send every write involving an account's history to each of the shards. It has the downside that queries by destination account will still be sent to all shards. The ledger remains with one PostgreSQL primary, because it is better to have it refused than it is to have it incorrect as the balance, and the MongoDB copy can trail. The failover test was conducted one machine with one node off at a time.

Evidence

The scripts and the screenshots are contained in each one of the following folders: portfolio-1-database-design, portfolio-2-query-optimisation, portfolio-3-transactions-concurrency, portfolio-4-nosql and portfolio-5-distributed-cloud (one folder per portfolio).

Individual reflection

The design decision I came back to most was the session ID. A UNIQUE constraint on each transfer table was sufficient, but won't prevent the same ID from showing up in two tables, so I added a registry table that every transfer is required to register in first. I also changed the column names, such that every table is named by the account number of an account, making it much easier to read the data when I did this by hand to check that results were correct.

I learned the most in my Portfolio 3. First checking a balance and then pushing a withdrawal in the same transaction is not safe under READ COMMITTED, and the second push of the account down to a negative number demonstrated this point. The lock resolved this with its second session waiting. In Portfolio 2 I found no differences in one query when I added the covering index – it didn't help for that query. I would add an index for the top-accounts pipeline to MongoDB and test SERIALIZABLE were I to have more time. I should also note that, with a full-time job, this was accomplished in a brief timeframe of heavy focus. I took some time off my work to make this happen.

References

PostgreSQL Global Development Group. PostgreSQL Documentation: Explicit Locking. https://www.postgresql.org/docs/current/explicit-locking.html

PostgreSQL Global Development Group. PostgreSQL Documentation: Using EXPLAIN. https://www.postgresql.org/docs/current/using-explain.html

PostgreSQL Global Development Group. PostgreSQL Documentation: Transaction Isolation. https://www.postgresql.org/docs/current/transaction-iso.html

MongoDB. Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/

MongoDB. Replication. https://www.mongodb.com/docs/manual/replication/

MongoDB. Sharding. https://www.mongodb.com/docs/manual/sharding/

MongoDB. Specify JSON Schema Validation. https://www.mongodb.com/docs/manual/core/schema-validation/specify-json-schema/

AI-use declaration

I utilized Anthropic's Claude primarily for research and to draft some of the SQL and test scripts to accelerate development work. It helped explain technical concepts as I progressed and assisted in generating supplementary materials like READMEs, MongoDB scripts, and the scripts used to render the ER and schema diagrams via diagrams.net and Graphviz.

However, I personally executed the core query, transaction, MongoDB, and Docker commands using pgAdmin and PowerShell. All database screenshots, query execution plans, timings, record counts, and test results represent the actual outputs generated on my local machine. I made all the architectural and design decisions detailed in my reflection, directed the iterative changes, and take full responsibility for the work as submitted.